



UPPSALA
UNIVERSITET

Image based Deep Learning for ECG diagnosis

Fernando Gallardo Garcia, Albion Rexhaj

Project in Computational Science: Report

April 1, 2026

PROJECT REPORT



Abstract

Cardiovascular diseases are the leading cause of death worldwide, and the Electrocardiogram (ECG) remains a cornerstone of their diagnosis. In recent years, several deep learning models have been developed capable of performing such diagnosis with high accuracy. Most of these however, rely on ECG-signals as their input. In practice, this format is rarely available for patients, as most medical centers usually only store (and provide) ECG images.

To address this issue, we present an algorithm that can perform multi-disease predictions based on ECG images. The model was trained using a dataset known as PTB-XL, consisting of ECG signals, which we then converted to images with specialized code. For performance comparison, we also trained a signal based model with the original dataset, using it as an ideal reference. In the end, we further increased the image model’s performance through the implementation of knowledge distillation, where the trained signal model acted as a teacher. This method boosted the image based classification performance by 0.5%–3% across all metrics on unseen data. Notably, recall, which measures a model’s ability to correctly identify positive diagnoses and is therefore one of the most important metrics in medical settings, showed one of the largest improvements over the baseline, with an average increase of 2.8% across the different labels. While these improvements are modest in absolute terms, they correspond to gap reductions between the image and signal models of 38%–100%, indicating highly effective distillation.

In summary, we developed an image based ECG multi-disease predictor that achieves performance comparable to its signal based counterpart. Knowledge distillation was instrumental in bridging this performance gap, enabling effective cross-modal knowledge transfer.

Contents

1	Introduction	3
2	Theoretical background	4
2.1	ECG structure	4
2.2	Deep learning models	5
2.3	ResNet	6
2.4	Validation metrics	8
2.5	Knowledge distillation	8
3	Methods	9
3.1	The PTB-XL Dataset	9
3.2	Generating ECG images from signals	10
3.3	Preprocessing	11
3.4	Cross-modal distillation	11
3.5	Model training	11
3.6	Threshold optimization	12
3.7	Performance evaluation and comparison	13
4	Results and discussion	13
5	Conclusions	20

1 Introduction

An ECG, or electrocardiogram, is a diagram of the heart’s electrical activity. It serves as a recording of the heartbeat by tracking the electrical signals in the heart and displaying it as a graph with various leads. A typical ECG is made up of twelve leads, which means that the electrical activity of the heart is monitored from twelve different perspectives using a total of ten different electrodes throughout the patient’s body. Some of the electrodes can be combined to give new leads, different from the original ones. The different leads together give doctors the ability to monitor the heart activity from multiple different angles, ensuring full coverage. These ECGs are crucial for hospitals to detect signs of a variety of different cardiovascular diseases, which are a leading cause of death worldwide [1].

In the last decade, deep learning has advanced substantially and become a very promising tool in identifying cardiovascular diseases. But the way they operate and make these predictions is still quite a mystery. The ways of interpreting ECGs is very different between deep learning algorithms and doctors, which adds to this uncertainty. It also makes the algorithms much less applicable in the real world, as they require completely different input data from doctors. While doctors use the diagrams and graphs as basis for interpreting ECGs, deep learning algorithms use the recorded electrical signals, which are completely unintelligible for a human. Therefore, an algorithm that uses the same input data as a human, meaning the ECG diagrams, would be a crucial tool in verifying a doctor’s prognosis. Also, an image based algorithm would be much more applicable in real world cases because ECGs are often saved as images instead of signals, and signals are often stored in a range of different formats making them quite incompatible.

This project aims to try and bring the different inputs closer together by creating a deep learning algorithm which predicts multi-label cases of cardiovascular diseases based on generated ECG images. This algorithm’s results will be compared to another algorithm that uses the recorded electrical signals, which allows us to interpret if this kind of algorithm is feasible for use in real world cases. The two algorithms will be trained on virtually the exact same dataset which will help us make an educated decision. We will also implement a machine learning technique called knowledge distillation, which allows to transfer performance from one model to another, to further increase the accuracy of the image model. For this, we will use the superior signal model as its teacher.

One of the main challenges of this project will be to handle the two different input formats and create algorithms that predict outputs accurately. This includes reasoning for different models, hyperparameters, and learning metrics, as well as preprocessing the data correctly. Much inspiration was taken from Digitization and Classification of ECG Images: The George B. Moody PhysioNet Challenge 2024 [12]. This was a public competition

held in 2024 to develop algorithms for classifying ECGs from images or paper printouts. Here, PhysioNet provides kits to generate images from ECG data, and cites multiple useful datasets to use as a starting point, one which we use in our project.

In this report, the theoretical background of ECGs and the used deep learning algorithms are described in §2, and methods regarding dataset and implementation are explained in §3. The results can be viewed in §4 and our conclusions are summarized in §5.

2 Theoretical background

This section describes the structure of ECGs and deep learning models, as well as the construction of image based models.

2.1 ECG structure

As stated in §1, ECGs are made up of twelve leads which give the full picture of the heart’s electrical activity. We can see these leads in the following figure of a typical ECG.

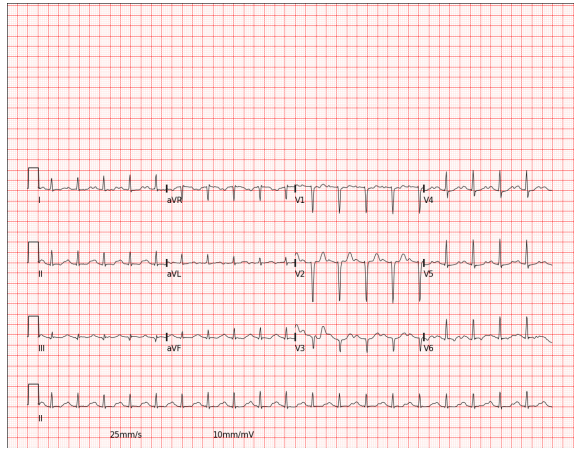


Figure 2.1: Generated image of typical ECG from dataset.

Figure 2.1 shows a generated ECG. The twelve leads are divided into three rows with four leads each. The last row is a longer recording of a specific lead, which is normally chosen to be lead II because it shows the most information individually. This project uses these generated images for the training process. These images are generated from raw ECG data using a toolkit for ECG image generator called 'ECG-Image-Kit' and are the input data we use in the image based model [3, 14]. The resolution we use varies and the images are converted to monochrome black and white.

2.2 Deep learning models

Deep learning is a branch of machine learning where algorithms are built up by multilayered neural networks. These neural networks mimic how neurons form connections and recognize patterns based on different weights and biases. Weights are flexible and determine how strongly different features and patterns of the input affect the final results, while biases are values that determine how sensitive the final result is to it. These networks then change and adapt the values of the weights and biases during training, in order to give accurate predictions [9, 10].

One method of constructing deep learning models is using PyTorch, which is a deep learning library in Python. This library has built in models and different network layers to be used in custom models. The final model used for the image based algorithm in this project was ResNet-50, a convolutional neural network (CNN) consisting of several different network layers and residual blocks. A diagram of how a typical 2D deep learning algorithm processes images can be seen in Figure 2.2.

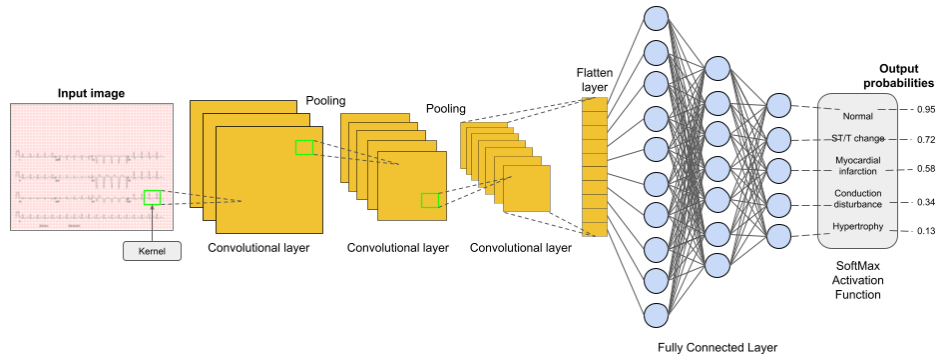


Figure 2.2: Typical image recognition algorithm using a Convolution Neural Network (CNN), which is a deep learning algorithm [2].

Figure 2.2 shows three different, most commonly used network layers; convolution, pooling, and fully connected. The ResNet-50 model consists of several of these different network layers. The most used one is a convolutional layer, which in the image based case is two-dimensional. These layers consists of a small window traveling across the input image, pixel-by-pixel and determining the scalar product of the values in the window, or kernel. This is done several times in order to detect distinct features in the image which is critical when generating predictions. These layers are the ones that find distinct edges and patterns in images and learns them by altering it's weights and biases, or trainable parameters.

The next important network layer is pooling layer. This layers' role is

to down sample or reduce the dimensions of the image and, subsequently reduce the number of parameters. The result of this is that information could be lost, but it helps massively with reducing computational complexity, improving computation time, improving robustness, and reducing the effects of overfitting. There are two versions of pooling layers, max pooling and average pooling. These work similarly to convolutional layers where a kernel travels across the input image. The difference here is that the max pooling layer selects the pixel with the largest value and disregards the rest, and the average pooling layer takes the average. Therefore, if we have a kernel with size 2×2 , the new pixel in the new output image will be the maximum value of those four previous ones, or the average. That means that the new image will have one fourth of the input images size.

The final important network layer is the fully connected layer. This layer focuses on interpreting the features generated by the previous convolutional and pooling layers, and producing predictions. Every sublayer in the fully connected layer consists of a number of neurons where each one is connected to all the previous ones, as well as all the next ones (see Figure 2.2 for visualization). Each connection has a weight and each neuron has a bias, as previously discussed [2].

2.3 ResNet

The ResNet model uses a combination of convolutional, pooling and fully connected layers to interpret input images and producing usable predictions. ResNet is based on the VGG-models, which is a CNN with a similar structure to the visualization in Figure 2.2. Several convolutional layers develop feature maps, meaning maps of distinct edges and patterns, the pooling layers reduce complexity and overfitting, and the fully connected layers interprets the feature map and learns in order to finally give predictions. ResNet models can vary in complexity, which is what the number '50' refers to in our ResNet-50 model. It tells us about the number of layers in the model that have trainable parameters, which in our case is 49 convolutional layers and 1 fully connected layer. The key difference from VGG models is the addition of "residual connections".

A problem that traditional CNNs face is that we can not make them as deep and complex as we want, as they will eventually start degrading and achieving worse results. This is because of a problem called vanishing and exploding gradients. These gradients are the signals that tell the model how to adjust its parameters for every training iteration. If the model has too many layers, what happens is that for each layer the gradient passes through it multiplies. When the model then has too many layers, the gradient will multiply several times leading to the signal either vanishing, meaning no changes in parameters for subsequent iterations, or exploding, meaning extremely large changes in parameters [15].

Residual connections are used to mitigate the problem of vanishing and exploding gradients and significantly improving efficiency and accuracy. The reason this is so effective, as with many techniques in deep learning, is still not fully understood. The idea is that the input images are not only passed through the layers like normal, but they are also directed around groups of layers. The two paths are then summed up element wise at chosen points. This is visualized in Figure 2.3. It might be a relatively simple trick but the effects are substantial because we achieve better results by training fewer parameters.

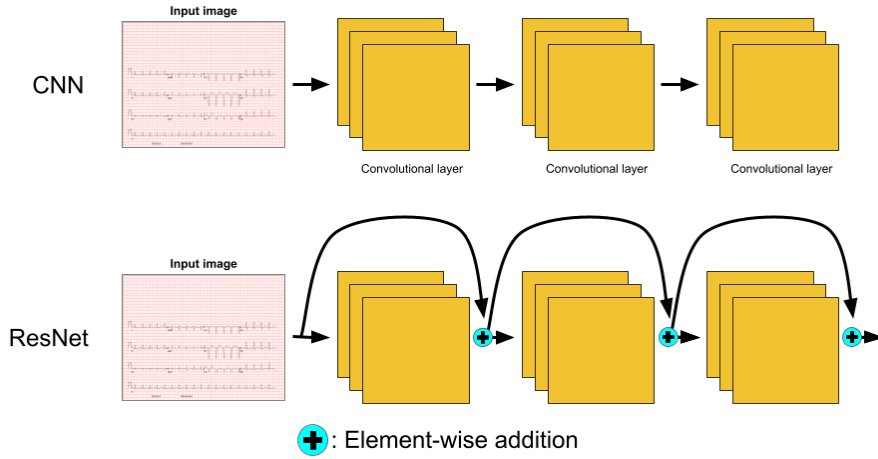


Figure 2.3: Visualization of residual connections in ResNet [16].

At the end of the fully connected layer, all the learned information and features are passed through a sigmoid function. This function converts the raw output scores (logits) into a probability distribution, like at the end of the CNN in Figure 2.2. It is important to point out that the data our model is trained on, the PTB-XL dataset is multi-label classification. This means that one patient can have multiple diseases or labels from the five previously discussed. Therefore instead of giving a probability distribution, such as in Figure 2.2 where the model is 70% sure the image shows a zebra, our model gives one probability for each class. For example for one image the normal label can have 78% probability while hypertrophy has 2% probability, and similar for the other labels. This means that we have to choose the threshold for our model to decide when to classify a label as true and when to classify them as false. The methodology for deciding this is discussed in §3.6.

2.4 Validation metrics

During training, we use the macro AUROC score as validation metric. AUROC, or Area Under the Receiver Operating Characteristic, is represented by drawing a graph with true positive rate (TPR) and false positive rate (FPR) on each axis, and calculating them for every possible threshold, then taking the area under. Therefore an AUROC score of 0.5 means random guessing, while an AUROC score of 1.0 means perfect results [4].

2.5 Knowledge distillation

To enhance our image model’s performance, we employ knowledge distillation[7], a technique often used to transfer learned representations from a larger, more capable model (the “teacher”) to a smaller, more efficient one (the “student”). While originally devised to address the trade-off between model capacity and computational efficiency, in this work we adapt knowledge distillation to transfer knowledge across input modalities (signals to images), as detailed in §3. This application is known as cross-modal distillation [5].

Conventionally, knowledge distillation mitigates a fundamental design compromise: large models often achieve strong performance due to high representational capacity, but at the cost of increased computational demands for both training and inference. Conversely, compact models are efficient but may lack the expressive power to capture complex patterns directly from data. Knowledge distillation bridges this gap by training the student model not only on the original task data, but also on the softened outputs or intermediate feature representations of the teacher, thereby transferring a more nuanced and generalizable understanding of the task.

More specifically, given a model $\mathbf{y}$ as a function of the vector variable $\mathbf{x}$ trained for a specific classification task, typically the final layer is a *softmax* in the form:

$$y_i(x | T) = \frac{e^{z_i(x)/T}}{\sum_{j \in C} e^{z_j(x)/T}} \quad (2.1)$$

where C denotes the set of the output classes, $z_i(\mathbf{x})$ is the logit for class i , and $T \geq 1$ is the temperature parameter (with $T = 1$ corresponding to the standard softmax). This transformation yields a “soft” probability distribution over classes.

Knowledge distillation trains the student using a composite loss $\mathcal{L}'$ that incorporates an additional cross-entropy term $\mathcal{L}_{KD}$ between the softened outputs of teacher $\hat{y}_i(\mathbf{x}|T)$ and student $y_i(\mathbf{x}|T)$:

$$\mathcal{L}_{KD}(\mathbf{x}|T) = - \sum_i \hat{y}_i(\mathbf{x}|T) \log y_i(\mathbf{x}|T), \quad (2.2)$$

where T acts as a hyperparameter. The teacher’s softened predictions provide a richer supervisory signal than hard labels, encapsulating relative class similarities learned during its training. Minimizing $\mathcal{L}_{\text{KD}}$ encourages the student to mimic the teacher’s internal representation, often leading to improved generalization. The temperature T controls the entropy of the output distribution; higher values produce “softer” probabilities that convey more relational information, while lower values sharpen the distribution toward one-hot encodings.

The complete training objective combines the original task’s loss $\mathcal{L}$ (e.g., cross-entropy with ground-truth labels $\bar{y}_i$) and the distillation loss:

$$\mathcal{L}'(\mathbf{x}|T) = \alpha\mathcal{L}(\mathbf{x}, \bar{\mathbf{y}}) + \beta T^2 \mathcal{L}_{\text{KD}}(\mathbf{x}|T). \quad (2.3)$$

Here α and β are hyperparameters that balance the two terms, and the factor T^2 compensates for the scaling of gradients induced by the temperature in the softmax derivatives.

3 Methods

As stated in §1, several deep learning models have been developed for the detection of cardiovascular diseases based on ECGs [6, 8, 13]. However, most of these rely on ECG-signal inputs, a format rarely available in practice. Therefore, we present a (multi-label) disease predictor based on ECG images, which are more commonly used. Our main objective is to achieve a similar performance to that of signal based models.

In order to perform a fair comparison, we designed and trained our model in parallel with a signal based one over the same dataset, PTB-XL. However, since this dataset is composed of ECG signal files instead of images, we first converted them PNG files using the ECG image kit [3] from [14]. For both versions (image and signal), we combined the respective input files into HDF5 files for computation efficiency during the training. The outputs were stored as binary arrays of length 5, corresponding to the 5 different classes. For the signal model, we used the architecture from [11] known as ResNet-1D with the default parameters. For the image-based approach, we trained several variants of the standard ResNet architecture with 18, 50, 101, and 152 layers. Since increasing depth beyond 50 layers did not yield additional performance gains, we selected ResNet-50. Finally, we applied knowledge distillation to further improve the image model, using the signal-based model as the teacher.

3.1 The PTB-XL Dataset

We used the PTB-XL dataset to train both of our models. This dataset consists of 21799 clinical 12-lead ECG signals from 18869 patients of 10

second length, stored as waveform files. Each of these is labeled with a set of possible ECG-diagnoses, which can be divided into five main categories:

- Normal
- ST/T Change
- Myocardial infarction
- Conduction disturbance
- Hypertrophy

The waveform files are stored in WaveForm DataBase (WFDB) format with 16 bit precision at a resolution of $1\mu V/LSB$ and a sampling frequency of 500Hz, as well as a downsampled version of 100Hz. Following our supervisor’s recommendation, we utilized the downsampled version as the model input. The data was split into training, validation and test according to the PTB-XL website’s recommendation as a 80/10/10 split. Their recommended validation and test sets underwent at least one human evaluation which makes them particularly reliable [17, 18].

3.2 Generating ECG images from signals

As stated in §3, we used the ECG image kit to generate the ECG images from the raw signal data [3, 14]. The code offers options to specify image resolution and other ECG specific characteristics. It also provides several options suited for data augmentation, a technique used in machine and deep learning that consists in artificially expanding the size and diversity of a training dataset by creating modified versions of existing data. This can be used to improve generalization, robustness (i.e. less sensitivity to variations), and to handle class imbalances. In our case, we had general image data augmentation techniques, such as randomly adding noise, rotations, crops, and color shifts, as well as ECG specific variations, such as altering the text appearing in the images or even adding wrinkles to mimic real world pictures. However, although data augmentation is a very effective tool for improving model performance in the field, it is also very computationally costly for the training, as it greatly increases the dataset size, and did therefore not fit in the scope of the project.

For this reason, we created images without distortions and with a relatively low resolution of (330 x 225), as we considered it provided a sensible tradeoff between model performance and training time. An example of a generated image can be viewed in Figure 2.1.

3.3 Preprocessing

For the signal based model, the following preprocessing steps were taken.

- Resampling all ECG traces to the sample sampling period (400 Hz).
- Zero padding if necessary such that all ECG have the same number of samples (4096).
- Removing trends in the ECG signal.

For the image based model, we converted the images to monochrome (i.e. 2D matrices where the value of each component corresponds to a pixel’s brightness) and removed the top 27.5%, as it contains no relevant information from the leads (See Figure 2.1). Additionally, its values to 1.

3.4 Cross-modal distillation

As stated in §2.5, after independently training the models, we employed knowledge distillation (in particular, cross-modal distillation) to further improve the image-based’s performance. For this, we used the signal-based model (the teacher) to transfer knowledge to the image-based (the student). This is possible because although the two models operate on distinct input modalities (signal vectors and image arrays) they perform the same classification task, as input pair constitute a different representation of the same underlying data. Furthermore, because the image representation is higher-dimensional and more complex, the signal-based teacher is expected to achieve substantially better performance when trained on identical data. By leveraging the paired nature of our dataset (for each image we have a corresponding signal representation), we computed the distillation loss between the teacher’s and student’s outputs for every matched sample using (2.3), thereby enabling knowledge distillation across modalities.

3.5 Model training

As stated previously, for each input type we first preprocess the data and then group it into two HDF5 files, corresponding to the training and testing sets. This approach improves computational efficiency by storing the data as contiguous arrays in memory, which reduces disk I/O overhead and enables faster data loading during training and evaluation. We then load the input data into the respective models in order to train them. The images used in the image model were generated and preprocessed before training, while the signals were preprocessed as previously described.

As mentioned in §3, for the signal model’s architecture we used ResNet-1D with default parameters, whereas for the image-based approach, we trained several variants of the standard ResNet architecture with 18, 50,

101, and 152 layers. Since increasing depth beyond 50 layers did not yield additional performance gains, we ended up selecting ResNet-50. Afterwards, we tuned the hyperparameters for both models. The parameters with the largest effect were learning rate, weight decay, and learning rate scheduler. Finally, we also trained and tuned the image model using KD.

The best-performing configurations are summarized in Table 3.1.

Hyperparameter	Signal	Image (No KD)	Image (KD)
Learning rate	10^{-3}	10^{-3}	10^{-3}
Weight decay	7×10^{-4}	10^{-5}	10^{-5}
LR scheduler	StepLR (step = 15, $\gamma = 0.5$)	None	None
Optimizer	Adam	Adam	AdamW
KD: (α, β, T)	–	–	(1, 1, 3)

Table 3.1: Optimization configurations for the signal model and the image models with and without knowledge distillation (KD).

Our choice of loss function for the image model was PyTorchs’ BCE-WithLogitsLoss function as the predefined architecture did not have a separate application of a Sigmoid layer, which is needed to generate probability distributions.

3.6 Threshold optimization

After finishing the training of the models, we get probabilities for each label. These are metrics on how sure the model is on classifying the labels. For example, the “normal” label could have 0.95 probability while “hypertrophy” has 0.12 probability. This means that this data point is probably normal and not hypertrophy. But how to distinguish a positive guess vs a negative one is up to us, and this is where the threshold optimization is implemented.

Instead of manually deciding on an arbitrary cut-off point for classification, like f.e. 0.5 which is commonly used, we go through each label in the solution of the validation set and test different thresholds. We test thresholds between 0.01 – 0.99 for each label individually and maximize their F1-scores. This gives each label a separate threshold and maximizes performance of all the models across all the metrics. An example of this can be seen in the following figure, which shows the thresholds for our image model.

Label	Thresholds
Normal	0.24
ST/T change	0.38
Myocardial infarction	0.55
Conduction disturbance	0.41
Hypertrophy	0.37

Table 3.2: Optimized thresholds for image model.

3.7 Performance evaluation and comparison

After performing hyperparameter tuning for each model and training configuration, that is, for the signal based, the image based and the image based using knowledge distillation (KD), we compared their performance on a test set of roughly 2000 samples.

Since the predictive ability of all models is dependent on the dataset size, we primarily focused on the relative performance between models, using the signal based as a reference of ideal performance.

One of the main goals in our analysis is to determine whether KD decisively improves performance rather than skewing toward certain metric, or the results being due to statistical variations in the test or during training. For this, we compared several threshold independent metrics, namely our BCE (our loss function), the macro Brier score and the AUROC, both micro and macro.

Secondly, we want to measure the extent of this increase in performance as well as to assess actual task performance. Therefore, we measured additional metrics such as precision, recall, specificity, F1-score, accuracy and balanced accuracy. All of these were calculated both for micro and macro averages. Finally, we derived the relative changes in these metrics introduced by KD, as well as the gap reduction between the signal based and the original image model.

4 Results and discussion

This section presents the final results of the models: the signal, image with no KD, and KD. Figures 4.1, 4.2 and 4.3 display the training runs that led to the best performing models for each configuration. Each figure consists of three plots showing, from top to bottom, the validation macro AUROC, the validation loss, and the training loss as functions of the training epoch. These curves are obtained by logging the corresponding metrics at the end of each epoch during training. In the AUROC plot, the red marker highlights the epoch achieving the highest validation macro AUROC; the corresponding model checkpoint was selected as the best-performing model for each

configuration. Additionally, Figure 4.4 displays all three validation AUROC curves for comparison.

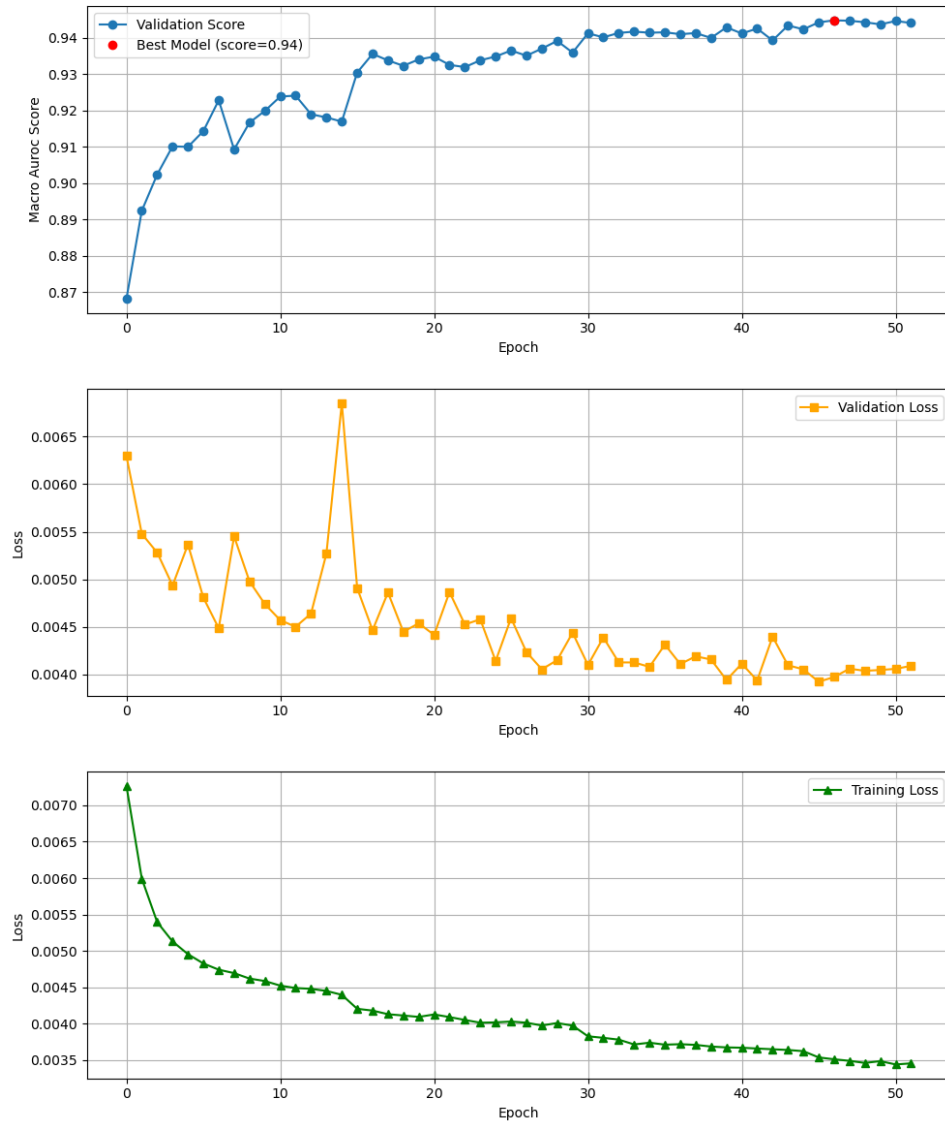


Figure 4.1: Evolution of training loss, validation loss and validation AUROC score during the training for the signal model.

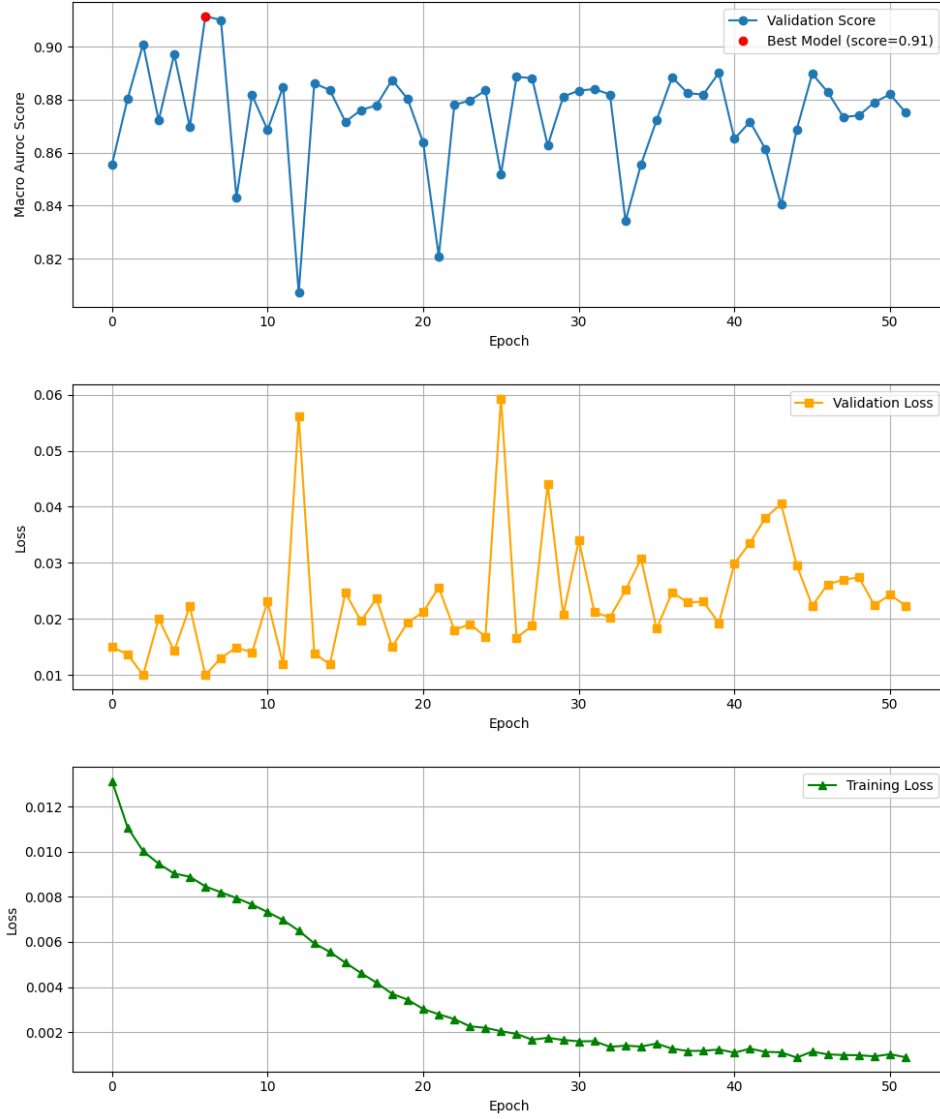


Figure 4.2: Evolution of training loss, validation loss and validation AUROC score during the training for the Image model without using KD. The validation loss is very noisy and shows an upward trend after epoch 20, indicating a very early overfitting. For that reason, the best performing model was also achieved during the first iterations. Although the noisiness of the validation curve is quite high when compared to the signal, mostly due to a high learning rate relative to the model's complexity, this high variance frequently lead to slightly better results than when using learning schedulers or lower learning rates.

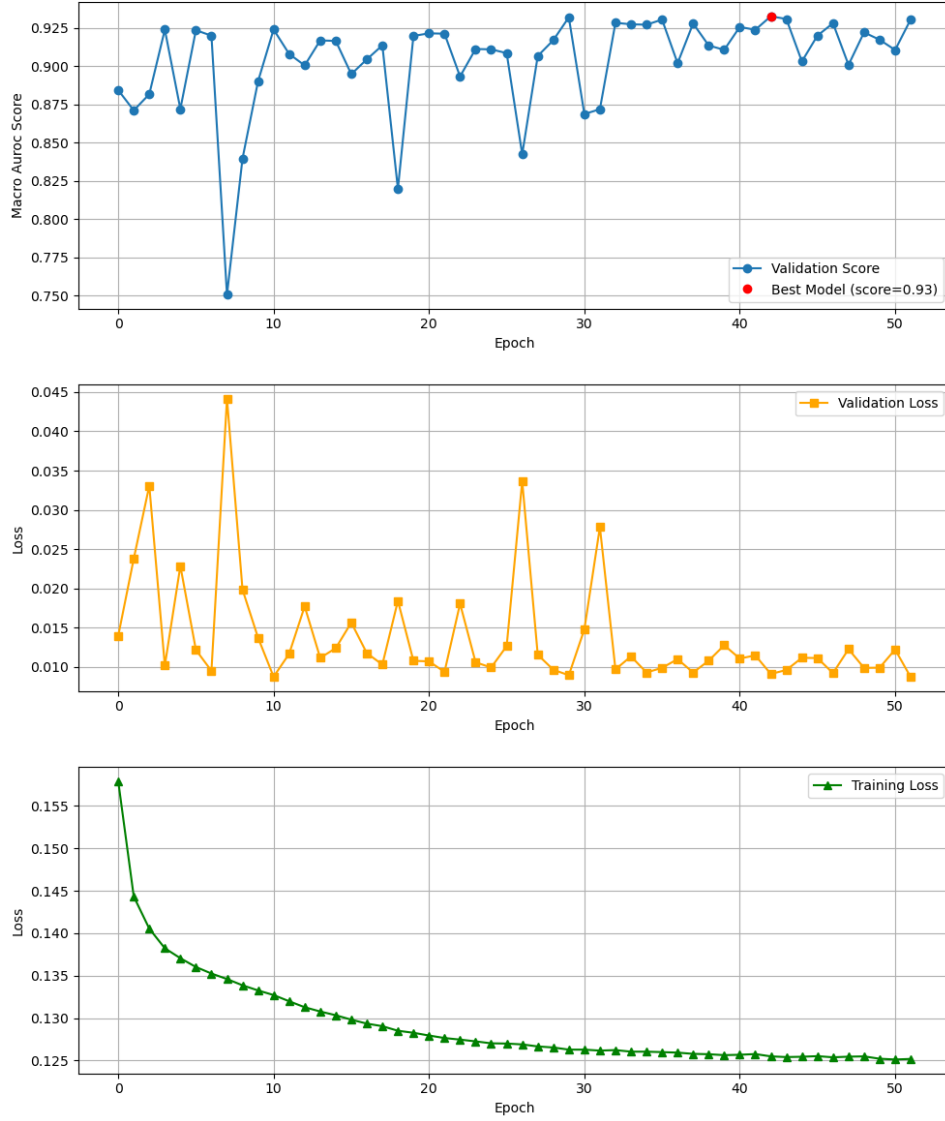


Figure 4.3: Evolution of training loss, validation loss and validation AUROC score during the training for the Image model using KD. In this case, The validation loss decreases more steadily and for longer than without KD. This is because the modified loss function indirectly reduces the variance of the gradients, allowing for higher learning rates and leading to a smoother convergence.

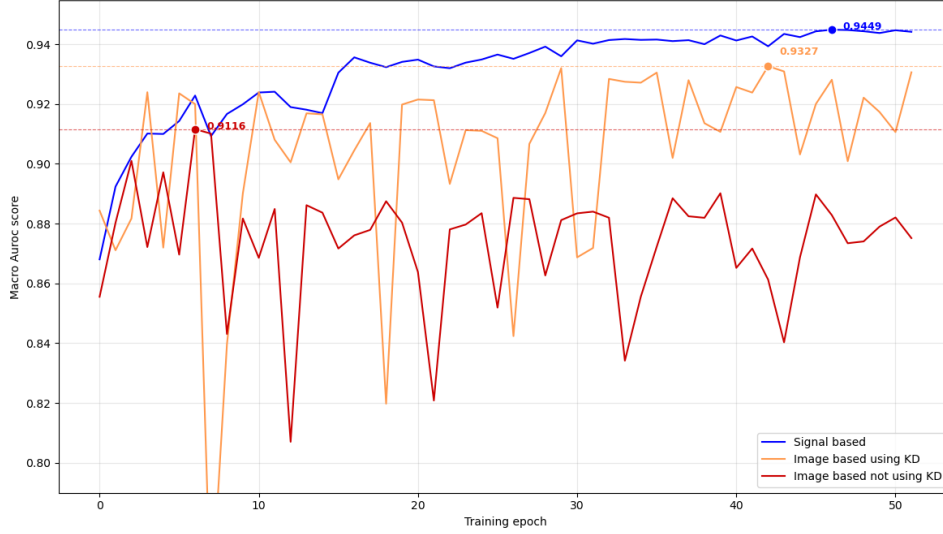


Figure 4.4: Evolution of the macro AUROC score during training between the three models. The highlighted dots indicate the best performing models for each configuration, which were the ones we used in the end. As it can be seen, using KD significantly reduces the performance gap between the signal and image.

As mentioned in §3.7, in order to determine whether KD decisively yielded an improvement in performance, we computed several non-threshold based metrics, listed in Table 4.1

Metric	Signal	KD	No KD
BCE Loss	0.307	0.323	0.327
Micro AUROC	0.937	0.929	0.909
Macro AUROC	0.928	0.917	0.900
Macro Brier Score	0.085	0.091	0.099

Table 4.1: Comparison of overall loss, discrimination (AUROC), and calibration (Brier) metrics on the test set.

As evidenced by the experimental results, knowledge distillation (KD) yielded improvements across all evaluated performance metrics. While the reduction in Binary Cross-Entropy (BCE) loss was relatively marginal, decreasing from 0.327 to 0.323 (a 1.1% improvement), the gain is less pronounced when contrasted with the performance of the signal (teacher) model. However, improvements in both the Brier score and AUROC were more substantial. The Brier score showed a consistent decrease across all diagnostic categories, whereas the AUROC (our target score during training) also

demonstrated marked gains, with the macro AUROC increasing by 1.9% and the micro AUROC by 2.2%. These improvements effectively bridged the performance gap between the image-based and the signal-based models by 61.5% and 66.3%, respectively. Given the consistent advancements across both macro and micro averages, we conclude that KD decisively enhances the predictive capabilities and reliability of the model.

Finally, we also measured threshold-dependent metrics on the test set in order to assess actual task performance, both for micro and macro averages. Results are shown in Figures 4.6 and 4.5, respectively. The relative improvements of KD over No KD, as well as the relative gap reduction of KD towards the signal model are listed in Tables 4.2 and 4.3.

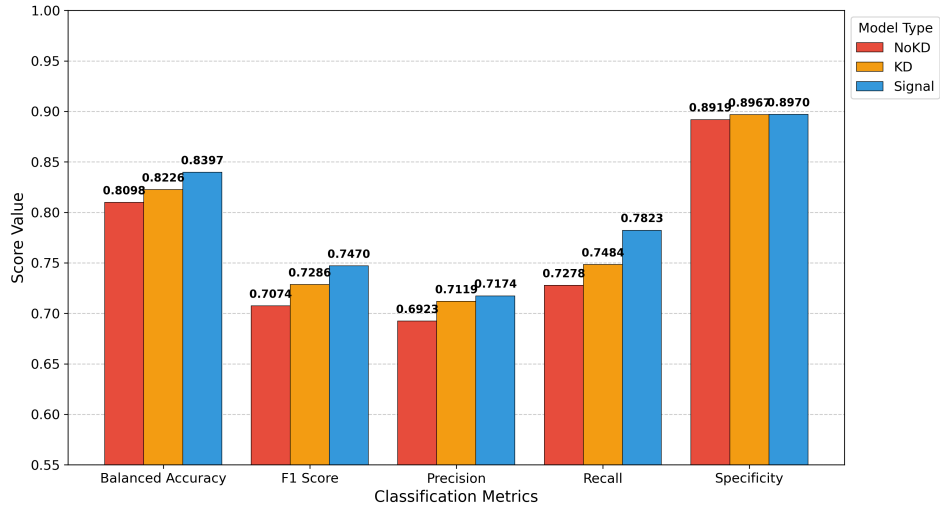


Figure 4.5: Comparison of Macro Average performance metrics across NoKD, KD, and signal models on the test.

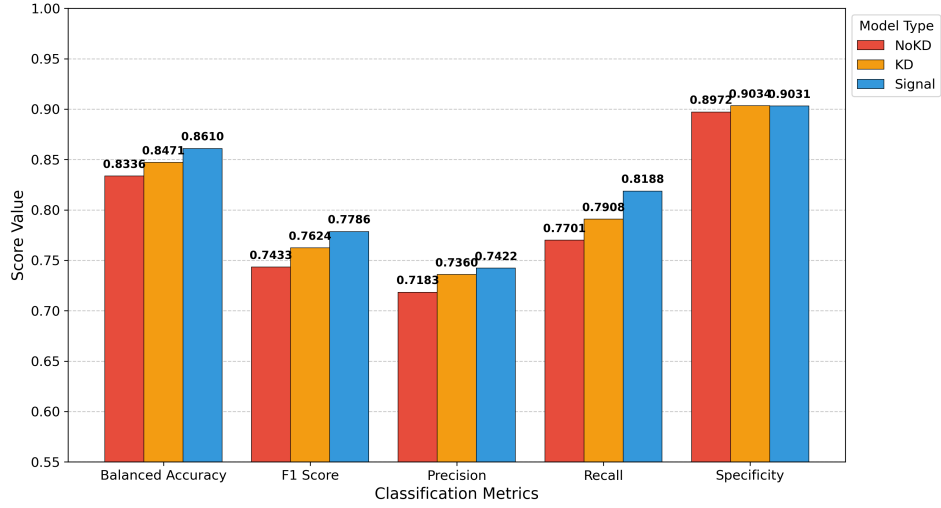


Figure 4.6: Comparison of Micro Average performance metrics across NoKD, KD, and signal models on the test.

Metric	Macro Improvement	Micro Improvement
AUROC	1.88 %	2.21 %
Accuracy	1.15 %	1.15 %
Balanced Accuracy	1.57 %	1.62 %
F1	2.99 %	2.57 %
NPV	0.89 %	0.78 %
Precision	2.84 %	2.46 %
Recall	2.83 %	2.70 %
Specificity	0.54 %	0.69 %

Table 4.2: Relative improvement of KD over NoKD (%).

Metric	Macro R-Gap Reduction	Micro R-Gap Reduction
AUROC	61.5 %	70.5 %
Accuracy	58.9 %	58.9 %
Balanced Accuracy	42.6 %	49.4 %
F1	53.5 %	54.2 %
NPV	46.4 %	44.1 %
Precision	78.3 %	74.0 %
Recall	37.8 %	42.6 %
Specificity	94.2 %	104.1 %

Table 4.3: Relative gap reduction of KD towards signal (%).

From Figures 4.5 and 4.6, we can see that there are improvements across all classification metrics as well. Even though these are not stark in absolute terms, Tables 4.2 and 4.3 show that KD decreases model differences by about 50% on average. Because improvements are consistent across metrics, both for the macro and micro averages, this confirms that KD does not merely skew the model toward favoring certain labels or metrics over others, but that generalization improves overall.

Regarding individual metrics, we observe that specificity had the highest relative gap reduction, with decreases of 94.2% and 104.1% for the macro and micro averages, respectively. This indicates that the signal model effectively transfers nearly all of its knowledge regarding the identification of negative diagnoses. The actual increase over the image baseline, however, was quite small (0.54% and 0.69%). On the other hand, recall (i.e., the ability to correctly identify true positives) is not transferred as effectively, with gap reductions of 37.8% and 42.6%. Nonetheless, the increase over the baseline was among the largest improvements in performance across all metrics (2.83% and 2.70%). This is especially significant given the importance of identifying positive diagnoses in medical settings.

5 Conclusions

In this section we discuss the main differences between the models, and their different results.

As stated in the introduction, we implemented an ECG image based deep learning model capable of performing multi-label diagnoses. The primary goal was to approach the performance of current state of the art signal based models. As we have seen, the signal based model produces noticeably better results as expected, but the image based model is still competitive, with F1 macro scores of 0.757 and 0.669 on the test set, respectively. In the end, we chose ResNet-101 as the architecture for the image based model, as it yielded better F1 scores than ResNet-50 while providing an acceptable

training time.

We observed that the learning rate was the most influential hyperparameter in performance. For the image model, a learning rate decreasing on plateaus proved to be effective convergence, whereas the image based was most successful when using a relatively high and fixed learning rate.

Although hyperparameter tuning did improve overall performance, the changes were not dramatic. The most influential aspect in the image model’s performance was KD, decreasing the gap between the image and signal models by around 50% for most metrics. In particular, it decreased the relative gap in macro AUROC, precision and recall by 61.5%, 78.3%, 37.8% respectively. Notably, the macro Recall experienced one of the largest improvements with respect to the baseline (2.8%), which is one of the most important metrics in medical settings.

Overall, the results have been very positive. The image model manages to sufficiently approximate the performance of the signal model, and could bridge the gap even further if trained on larger datasets, potentially allowing it to be of medical use in the real world. Future work could focus on expanding the knowledge distillation method to also connect inner layers between models instead of just the output, leading to a greater distillation. Additionally, one could also consider performing data augmentation by adding distortions and noise to the images.

Acknowledgment

We want to thank Antonio Horta Ribeiro for being a helpful and open supervisor, as well as Stefan Engblom for arranging this course with multiple helpful lectures.

Gemini 3 was used to generate helpful plots and as a text-editing tool for minor parts of the text.

References

- [1] Britannica. Electrocardiography, 2025. URL <https://www.britannica.com/science/electrocardiography>. Accessed 2025-12-13.
- [2] E. Canorea. Convolutional neural network: Guide to understanding them, No date. URL <https://www.plainconcepts.com/convolutional-neural-network-guide/>. Accessed 2026-01-02.
- [3] Deepanshi. ECG-image-kit: A toolkit for synthesis, analysis, and digitization of electrocardiogram images, 2024. URL <https://github.com/alphanumericlab/ecg-image-kit>. Accessed 2025-12-10.

- [4] Google. Classification: ROC and AUC, 2026. URL <https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc>. Accessed 2026-01-03.
- [5] S. Gupta, J. Hoffman, and J. Malik. Cross modal distillation for supervision transfer, 2015. URL <https://arxiv.org/abs/1507.00448>.
- [6] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi, G. H. Tison, C. Bourn, M. P. Turakhia, and A. Y. Ng. Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network. *Nature Medicine*, 25(1):65–69, 2019. ISSN 1546-170X. doi:10.1038/s41591-018-0268-3. URL <https://doi.org/10.1038/s41591-018-0268-3>.
- [7] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network, 2015. URL <https://arxiv.org/abs/1503.02531>.
- [8] J. W. Hughes, J. E. Olgin, R. Avram, S. A. Abreau, T. Sittler, K. Radia, H. Hsia, T. Walters, B. Lee, J. E. Gonzalez, and G. H. Tison. Performance of a convolutional neural network and explainability technique for 12-lead electrocardiogram interpretation. *JAMA Cardiology*, 6(11):1285–1295, 11 2021. ISSN 2380-6583. doi:10.1001/jamacardio.2021.2746. URL <https://doi.org/10.1001/jamacardio.2021.2746>. Accessed 2026-12-01.
- [9] IBM. What is a neural network? URL <https://www.ibm.com/think/topics/neural-networks>. Accessed 2026-01-02.
- [10] IBM. What is deep learning?, No date. URL <https://www.ibm.com/think/topics/deep-learning>. Accessed 2026-01-02.
- [11] E. M. Lima, A. H. Ribeiro, G. M. Paixão, M. H. Ribeiro, M. M. P. Filho, P. R. Gomes, D. M. Oliveira, E. C. Sabino, B. B. Duncan, L. Gatti, S. M. Barreto, W. Meira, T. B. Schön, and A. L. P. Ribeiro. Deep neural network estimated electrocardiographic-age as a mortality predictor. *Nature Communications*, 12, 2021. doi:10.1038/s41467-021-25351-7. Accessed 2026-12-01.
- [12] PhysioNet. Digitization and classification of ECG images: The george b. moody physionet challenge 2024, 2024. URL <https://moody-challenge.physionet.org/2024/>. Accessed 2025-12-29.
- [13] A. H. Ribeiro, M. H. Ribeiro, G. M. M. Paixão, D. M. Oliveira, P. R. Gomes, J. A. Canazart, M. P. S. Ferreira, C. R. Andersson, P. W. Macfarlane, W. Meira Jr., T. B. Schön, and A. L. P. Ribeiro. Automatic diagnosis of the 12-lead ecg using a deep neural network. *Nature Communications*, 11(1):1760, 2020. ISSN 2041-1723. doi:10.1038/s41467-020-15432-4. URL <https://doi.org/10.1038/s41467-020-15432-4>.

- [14] K. Shivashankara, Deepanshi, A. Shervedani, G. Clifford, M. Reyna, and R. Sameni. ECG-image-kit: a synthetic image generation toolbox to facilitate deep learning-based electrocardiogram digitization, 2024. URL <https://iopscience.iop.org/article/10.1088/1361-6579/ad4954>. Accessed 2025-12-10.
- [15] P. Subhraneel. Vanishing and exploding gradients problems in deep learning, 2025. URL <https://www.geeksforgeeks.org/deep-learning/vanishing-and-exploding-gradients-problems-in-deep-learning/>. Accessed 2026-01-02.
- [16] A. Vina. What is ResNet-50 and what is its relevance in computer vision?, 2025. URL <https://www.ultralytics.com/blog/what-is-resnet-50-and-what-is-its-relevance-in-computer-vision>. Accessed 2026-01-02.
- [17] P. Wagner, N. Strodthoff, R.-D. Bousseljot, D. Kreiseler, F. I. Lunze, W. Samek, and T. Schaeffter. PTB-XL, a large publicly available electrocardiography dataset. *Scientific Data*, 7(1):154, 2020. ISSN 2052-4463. doi:[10.1038/s41597-020-0495-6](https://doi.org/10.1038/s41597-020-0495-6).
- [18] P. Wagner, N. Strodthoff, R. Bousseljot, W. Samek, and T. Schaeffter. PTB-XL, a large publicly available electrocardiography dataset, 2022. RRID: SCR_007345.